



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

4

Estrutura do Projeto, Empacotamento e Testes

Resumo

Nesta unidade abordamos práticas essenciais para a organização e evolução de aplicações web desenvolvidas com Flask. Inicialmente discutimos como transformar um projeto em um pacote Python instalável, utilizando o arquivo `setup.py` e o gerenciador de pacotes `pip`. Em seguida, exploramos estratégias para gerenciamento de dependências com arquivos `pyproject.toml`. Também apresentamos o uso do *invoke* para automação de tarefas recorrentes no processo de desenvolvimento. Por fim, introduzimos conceitos fundamentais de testes automatizados utilizando o framework `PyTest` e sua extensão `PyTest-Flask`, demonstrando como estruturar testes para aplicações web e verificar automaticamente o funcionamento do sistema.

Palavras-chave: Flask, empacotamento Python, `pip`, `pytest`, testes automatizados, engenharia de software.

*“Programas devem ser escritos para pessoas lerem, e apenas
incidentalmente para máquinas executarem.”*

Harold Abelson

Introdução

À medida que aplicações web evoluem, torna-se fundamental organizar adequadamente a estrutura do projeto. Sistemas maiores exigem práticas que facilitem manutenção, reutilização de código e implantação em diferentes ambientes.

Em projetos simples, é comum executar o aplicativo diretamente a partir do diretório local. Entretanto, quando a aplicação cresce ou precisa ser implantada em servidores, *pipelines* de integração contínua ou ambientes de testes automatizados, torna-se necessário estruturar o projeto como um **pacote Python instalável**.

Transformar a aplicação em um pacote instalável traz diversas vantagens. Entre elas, destacam-se:

- permitir que o projeto seja instalado em qualquer ambiente Python;
- facilitar a execução de testes automatizados;
- simplificar processos de deploy;
- garantir que dependências sejam corretamente resolvidas.

Além da organização do projeto, outro aspecto fundamental no desenvolvimento profissional de software é a criação de **testes automatizados**. Testes permitem verificar continuamente se o sistema continua funcionando corretamente à medida que novas funcionalidades são adicionadas.

Nesta unidade, portanto, serão apresentados três pilares importantes da engenharia de software aplicada ao desenvolvimento web com Flask:

1. Empacotamento de aplicações Python;
2. Gerenciamento de dependências;
3. Testes automatizados utilizando PyTest.

Transformando a Aplicação em um Pacote Python

No ecossistema Python, módulos e bibliotecas são distribuídos na forma de **pacotes**. Esses pacotes podem ser instalados utilizando a ferramenta `pip`, que os coloca em um diretório especial chamado *site-packages*.

Esse diretório faz parte do `sys.path`, que representa os caminhos onde o Python procura módulos quando utilizamos a instrução `import`.

Considere o exemplo abaixo:

```
1 import sys
2 print(sys.path)
```

O Python percorre esses diretórios sempre que um módulo é importado. Portanto, para que um projeto possa ser utilizado em diferentes ambientes, é conveniente que ele também esteja disponível nesses caminhos.

Para tornar uma aplicação instalável, utilizamos um arquivo chamado `setup.py`, responsável por definir os metadados e informações necessárias para a instalação do pacote.

Empacotamento moderno da aplicação

Nos exemplos introdutórios de empacotamento em Python é comum encontrar a utilização do arquivo `setup.py`. Historicamente, esse arquivo era responsável por definir metadados do projeto, dependências e o mecanismo de construção do pacote. No entanto, nas práticas modernas de desenvolvimento Python, esse modelo tem sido gradualmente substituído por uma abordagem padronizada baseada no arquivo `pyproject.toml`.

Essa mudança foi formalizada por meio de diferentes propostas de melhoria da linguagem (Python Enhancement Proposals – PEPs), em especial a **PEP 517**, **PEP 518** e a **PEP 621**. Essas especificações definem um padrão para descrever como um projeto Python deve ser construído e quais são seus metadados essenciais.

Neste curso adotaremos essa abordagem moderna. Em vez de utilizar um arquivo `setup.py`, nossa aplicação Flask será configurada a partir de um arquivo `pyproject.toml`, que cen-

traliza as informações de construção, dependências e metadados do projeto.

Motivações da escolha arquitetural

A adoção do `pyproject.toml` apresenta diversas vantagens em relação ao modelo tradicional:

- **Padronização moderna:** o arquivo segue as recomendações atuais da comunidade Python e das PEPs mais recentes.
- **Separação clara de responsabilidades:** o sistema de build é declarado explicitamente na seção `[build-system]`.
- **Melhor interoperabilidade:** diferentes ferramentas de build (como *setuptools*, *poetry* ou *flit*) podem utilizar o mesmo formato.
- **Organização centralizada:** metadados, dependências e configurações de ferramentas ficam em um único arquivo.
- **Reprodutibilidade do ambiente:** facilita a instalação da aplicação em diferentes ambientes de desenvolvimento ou produção.

Estrutura do arquivo `pyproject.toml`

O arquivo `pyproject.toml` é atualmente o mecanismo padronizado para descrever como um projeto Python deve ser construído e quais são seus metadados. Ele foi introduzido inicialmente pelas PEPs 517 e 518 e posteriormente expandido pela PEP 621, que definiu uma estrutura padronizada para as informações de projeto.

De forma geral, esse arquivo possui dois papéis principais:

- descrever **como o pacote deve ser construído** (processo de build);
- registrar **metadados do projeto e suas dependências**.

Assim, ferramentas como `pip`, `build` e sistemas de integração contínua podem automaticamente compreender como instalar e compilar o projeto.

Sistema de construção (Build System)

A primeira seção do arquivo define qual mecanismo será responsável pela construção do pacote Python.

```
1 [build-system]
2 requires = ["setuptools>=70.0", "wheel"]
3 build-backend = "setuptools.build_meta"
```

Essa seção possui dois elementos principais:

- `requires`: lista de dependências necessárias para executar o processo de construção do pacote. Nesse caso utilizamos o `setuptools` e o `wheel`.
- `build-backend`: define qual backend implementa o processo de build. O módulo `setuptools.build_meta` indica que o sistema de empacotamento utilizado será o `setuptools`.

Durante a instalação do pacote, ferramentas como o `pip` criam um ambiente isolado contendo essas dependências e executam o backend de build para gerar o pacote instalável.

Metadados do projeto

A seção `[project]` contém as informações descritivas da aplicação. Esses dados são utilizados por ferramentas de distribuição de software, repositórios de pacotes e sistemas de gerenciamento de dependências.

```
1 [project]
2 name = "delivery"
3 # major (mudanca grande), minor (versao trabalhada), patch (correcao bug)
4 version = "0.1.0"
5 description = "A web platform for managing food delivery orders and restaurant
6               operations."
7 readme = "README.md"
8 requires-python = ">=3.9"
9 license = {file = "LICENSE"}
10 authors = [
```

```
10     {name = "Wanderson Santana", email = "wanderson.santana@uvv.br"}
11 ]
12 dependencies = [
13     "Flask",
14 ]
```

Cada campo possui uma função específica:

- **name**: nome do pacote Python que será instalado no ambiente.
- **version**: versão atual do software. Neste projeto adotamos o modelo de *Semantic Versioning* (SemVer), composto por três números: **major.minor.patch**. Alterações estruturais incrementam a versão *major*, novas funcionalidades incrementam *minor* e correções de erros incrementam *patch*.
- **description**: breve descrição textual do propósito da aplicação.
- **readme**: arquivo que contém a documentação principal do projeto, normalmente exibida em repositórios como GitHub ou PyPI.
- **requires-python**: versão mínima do interpretador Python necessária para executar o projeto.
- **license**: especifica a licença de distribuição do software.
- **authors**: lista de autores responsáveis pelo projeto.
- **dependencies**: lista de bibliotecas externas necessárias para executar a aplicação. Durante a instalação do pacote essas dependências são automaticamente resolvidas e instaladas pelo gerenciador de pacotes.

Dependências opcionais

Além das dependências obrigatórias para execução do sistema, é comum definir conjuntos de dependências adicionais para diferentes cenários de uso do projeto.

```
1 [project.optional-dependencies]
2 dev = [...]
3 test = [...]
```

Nesse caso, são definidos dois grupos adicionais:

- **dev**: ferramentas utilizadas durante o desenvolvimento, como depuradores, formata-dores de código e utilitários de inspeção.
- **test**: bibliotecas necessárias para execução da suíte de testes automatizados da aplicação.

Essa separação é importante pois permite que ambientes de produção instalem apenas as dependências essenciais para execução do sistema, mantendo o ambiente mais enxuto e reduzindo a superfície de possíveis conflitos de dependência.

Por outro lado, desenvolvedores do projeto podem instalar todos os recursos adicionais com comandos como:

```
1 pip install -e .[dev,test]
```

Esse mecanismo facilita a reprodução do ambiente de desenvolvimento e torna o processo de colaboração em projetos de software mais padronizado e escalável.

Impacto na estrutura do projeto

Ao transformar nossa aplicação Flask em um pacote instalável utilizando o arquivo `pyproject.toml`, passamos a adotar uma organização de software alinhada com práticas utilizadas em projetos profissionais de desenvolvimento Python.

Essa abordagem modifica a forma como o código da aplicação é estruturado, instalado e executado. Em vez de tratar o projeto apenas como um conjunto de scripts executados diretamente, ele passa a ser organizado como um **pacote Python instalável**. Isso significa que a aplicação pode ser instalada no ambiente de execução da mesma forma que qualquer outra biblioteca do ecossistema Python.

Entre os principais benefícios dessa abordagem destacam-se:

- **Instalação do projeto no ambiente Python**

O projeto pode ser instalado diretamente no ambiente virtual utilizando o comando:


```
1 pip install -e .
```

O parâmetro `-e` indica uma instalação no modo *editable*. Nesse modo, o pacote é instalado no ambiente Python, mas permanece referenciando diretamente os arquivos do diretório de desenvolvimento. Isso permite que alterações no código sejam refletidas imediatamente sem necessidade de reinstalar o pacote.

- **Organização modular do código**

Ao estruturar o projeto como um pacote Python, o código passa a ser organizado em módulos e subpacotes bem definidos. Essa abordagem favorece princípios importantes de engenharia de software, como *modularidade*, *separação de responsabilidades* e *reutilização de componentes*.

Além disso, essa organização facilita a manutenção do código e a evolução da aplicação ao longo do tempo.

- **Gerenciamento explícito de dependências**

As dependências necessárias para execução da aplicação passam a ser declaradas explicitamente no arquivo `pyproject.toml`. Isso permite que ferramentas como `pip` resolvam automaticamente essas dependências durante a instalação do projeto.

Esse mecanismo contribui para a reprodutibilidade do ambiente de execução, reduzindo problemas relacionados a diferenças de versão entre bibliotecas utilizadas por diferentes desenvolvedores.

- **Facilidade de distribuição do software**

Uma vez estruturado como pacote Python, o projeto pode ser facilmente empacotado e distribuído em repositórios de software, como o *Python Package Index* (PyPI), ou em repositórios privados utilizados por equipes de desenvolvimento.

Isso permite que a aplicação seja instalada em diferentes ambientes (como servidores de produção ou containers) de maneira padronizada.

- **Integração com ferramentas de desenvolvimento**

Projetos estruturados dessa forma integram-se naturalmente com diversas ferramentas utilizadas no ciclo de desenvolvimento de software, como sistemas de testes

automatizados, analisadores estáticos de código, formatadores e plataformas de integração contínua (CI/CD).

Dessa forma, além de compreender os conceitos fundamentais do framework Flask e da construção de aplicações web, passamos também a trabalhar com práticas modernas de organização de projetos, empacotamento de software e gerenciamento de dependências no ecossistema Python.

Essas práticas aproximam o ambiente de aprendizado do contexto encontrado em projetos reais de desenvolvimento de software.

Instalação do Pacote

Após a definição dos metadados do projeto no arquivo `pyproject.toml`, a aplicação pode ser instalada no ambiente Python utilizando o gerenciador de pacotes `pip`.

Durante o desenvolvimento, a forma mais comum de instalação é o **modo editável** (*editable install*), que permite trabalhar diretamente sobre o código-fonte do projeto.

```
pip install -e .
```

Nesse modo, o pacote é registrado no ambiente Python, mas continua apontando para o diretório local do projeto. Assim, quaisquer alterações realizadas nos arquivos do código-fonte passam a ser imediatamente refletidas na aplicação, sem necessidade de reinstalar o pacote.

Internamente, o `pip` utiliza as informações definidas no `pyproject.toml` para identificar o sistema de construção (*build system*) do projeto e executar o backend responsável pelo empacotamento do software.

Ao executar o comando acima, o terminal apresentará uma sequência de mensagens descrevendo as etapas do processo de instalação. Um exemplo simplificado dessa saída pode ser observado a seguir:

```
pip install -e .
Obtaining file:///.../UVV/
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting Flask (from delivey==0.1.0)
...
Building editable for delivey (pyproject.toml) ... done
Successfully built delivey
Installing collected packages: Flask, delivey
Successfully installed Flask-3.1.3 delivey-0.1.0
```

Cada etapa dessa saída corresponde a uma fase do processo de instalação:

- **Obtaining:** indica que o pip está utilizando o diretório atual do projeto como origem do pacote.
- **Installing build dependencies:** instalação das dependências necessárias para executar o processo de construção definido no `pyproject.toml`.
- **Preparing editable metadata:** leitura dos metadados do projeto e preparação das informações necessárias para o empacotamento.
- **Collecting:** resolução e download das dependências externas definidas no projeto (neste caso, o framework Flask e suas bibliotecas auxiliares).
- **Building editable:** geração de um pacote instalável em modo editável.
- **Successfully installed:** confirmação de que o pacote do projeto e suas dependências foram instalados corretamente no ambiente Python.

Esse mecanismo facilita o ciclo de desenvolvimento, permitindo que o projeto seja tratado como um pacote Python completo, ao mesmo tempo em que preserva a flexibilidade necessária para modificações frequentes durante a implementação da aplicação.

Em projetos reais, essa forma de instalação também contribui para a padronização do ambiente de desenvolvimento entre diferentes membros da equipe.

Gerenciamento de Dependências

Projetos Python utilizam gerenciadores de pacotes para controlar as bibliotecas externas necessárias ao funcionamento do sistema. Historicamente, essas dependências eram registradas em arquivos `requirements.txt`. Entretanto, com a adoção do `pyproject.toml`, tornou-se possível declarar essas dependências diretamente na configuração do projeto.

As dependências podem ser classificadas em dois grupos principais:

- **Dependências de execução** — necessárias para que a aplicação funcione corretamente em ambiente de produção.
- **Dependências de desenvolvimento** — utilizadas apenas durante o processo de desenvolvimento do software, como ferramentas de formatação de código, análise estática e execução de testes.

No arquivo `pyproject.toml`, essas dependências adicionais podem ser organizadas utilizando a seção `project.optional-dependencies`:

```
1 [project.optional-dependencies]
2 dev = [
3     "ipython",
4     "black",
5     "flake8",
6 ]
7
8 test = [
9     "pytest >= 8.3",
10    "pytest-cov >= 5.0",
11    "pytest-flask",
12 ]
```

Essa organização permite instalar apenas os componentes necessários para cada contexto de uso do projeto. Por exemplo, um desenvolvedor pode instalar todas as ferramentas de desenvolvimento utilizando:

```
pip install -e .[dev]
```

Já em ambientes de produção, apenas as dependências essenciais serão instaladas, mantendo o ambiente mais enxuto e reduzindo possíveis conflitos entre bibliotecas.

Ferramentas de desenvolvimento

As dependências listadas acima correspondem a ferramentas amplamente utilizadas no ecossistema Python para apoio ao desenvolvimento de software.

- **IPython** é um interpretador interativo avançado para Python, que oferece recursos como autocompletar, inspeção de objetos e histórico de comandos. Ele é amplamente utilizado para experimentação e depuração de código [1].
- **Black** é um formatador automático de código Python. Seu objetivo é padronizar a formatação do código-fonte de acordo com regras consistentes, facilitando a leitura e reduzindo conflitos em revisões de código [2].
- **Flake8** é uma ferramenta de análise estática que verifica erros de estilo e possíveis problemas no código Python, combinando checagens de PEP8, PyFlakes e McCabe complexity [3].

Ferramentas de teste

Além das ferramentas de desenvolvimento, também definimos dependências relacionadas à execução de testes automatizados:

- **Pytest** é um framework de testes amplamente utilizado na comunidade Python, que permite escrever testes de forma simples e extensível [4].
- **pytest-cov** é uma extensão do Pytest que permite medir a cobertura de testes do código, indicando quais partes do sistema estão ou não sendo testadas [5].

- **pytest-flask** fornece utilidades específicas para testar aplicações desenvolvidas com o framework Flask [6].

A utilização dessas ferramentas contribui para a adoção de práticas modernas de engenharia de software, como integração contínua, qualidade de código e testes automatizados.

Automação de tarefas com Invoke

Projetos de software frequentemente possuem tarefas recorrentes, como instalação de dependências, execução de testes e inicialização da aplicação. Automatizar essas tarefas contribui para tornar o fluxo de desenvolvimento mais padronizado e reproduzível.

Em ambientes Unix é comum utilizar arquivos `Makefile` para esse propósito. Entretanto, em projetos Python modernos é bastante comum utilizar ferramentas de automação escritas na própria linguagem.

Uma dessas ferramentas é o **Invoke**, que permite definir tarefas automatizadas em um arquivo Python chamado `tasks.py`.

Cada função decorada com `@task` torna-se um comando executável a partir da linha de comando.

Exemplo de arquivo `tasks.py`

Um exemplo simplificado de automação para nosso projeto pode ser observado a seguir:

```
1 from invoke import task
2
3
4 @task
5 def install(c):
6     """Instala o projeto em modo editavel."""
7     c.run('pip install -e ".[dev,test]"', echo=True)
8
9
10 @task
```

```
11 def test(c):
12     """Executa os testes automatizados."""
13     c.run("pytest tests -v", echo=True)
14
15
16 @task
17 def run(c):
18     """Executa a aplicacao Flask."""
19     c.run("flask run", pty=True)
```

Após definir o arquivo `tasks.py`, os comandos podem ser executados utilizando o utilitário `invoke`.

Executando tarefas automatizadas

As tarefas definidas podem ser executadas da seguinte forma:

```
invoke install
invoke run
invoke test
invoke lint
invoke format
```

Essa abordagem apresenta algumas vantagens importantes:

- as tarefas são escritas em Python, facilitando sua manutenção;
- é possível adicionar parâmetros e lógica de controle (condicionais, loops, etc.);
- o mesmo sistema funciona em diferentes sistemas operacionais;
- novas tarefas podem ser adicionadas facilmente conforme o projeto evolui.

Dessa forma, o arquivo `tasks.py` passa a funcionar como um pequeno sistema de automação do projeto, simplificando tarefas frequentes no processo de desenvolvimento.

Testes Automatizados

Testes automatizados são fundamentais para garantir a qualidade de software. Eles permitem verificar automaticamente se partes do sistema continuam funcionando corretamente após modificações no código.

Ao escrever testes, o desenvolvedor descreve o comportamento esperado de determinadas funcionalidades do sistema. Sempre que o código é alterado, os testes podem ser executados novamente para verificar se esse comportamento continua correto.

Essa prática ajuda a prevenir regressões, facilita a refatoração do código e aumenta a confiabilidade da aplicação.

No ecossistema Python, uma das ferramentas mais populares para testes é o **PyTest**. O PyTest oferece uma sintaxe simples, suporte a testes parametrizados, fixtures e uma integração eficiente com diversas bibliotecas do ecossistema Python.

Estrutura de testes

Em muitos projetos Python, os testes são organizados em um diretório específico chamado `tests`. Um exemplo de estrutura de projeto é:

```
1 project/  
2 |--- src/  
3 |   |-- app.py  
4 |   |-- tests/  
5 |   |-- test_example.py  
6 |   |-- pyproject.toml
```

Arquivos de teste geralmente começam com o prefixo `test_`, permitindo que o PyTest os identifique automaticamente.

Escrevendo um teste simples

Um teste simples pode ser escrito da seguinte forma:


```
1 def test_sum():  
2     assert 1 + 1 == 2
```

Nesse exemplo, o comando `assert` verifica se a expressão é verdadeira. Caso contrário, o teste falha e o PyTest apresenta uma mensagem indicando o erro encontrado.

Quando executado, o PyTest analisa automaticamente os arquivos de teste e executa todas as funções cujo nome começa com `test_`.

Executando os testes

Para executar os testes do projeto, basta utilizar o seguinte comando:

```
1 pytest -v
```

A opção `-v` ativa o modo verboso, exibindo informações mais detalhadas sobre cada teste executado.

Medição de cobertura de testes

Além de verificar se os testes passam ou falham, também é possível medir quanto do código está sendo exercitado pelos testes. Para isso, utiliza-se o plugin `pytest-cov`.

Um exemplo de execução com análise de cobertura é:

```
1 pytest --cov=src
```

Esse comando gera um relatório indicando quais partes do código foram executadas durante os testes.

Testes em aplicações Flask

Quando se trabalha com aplicações web desenvolvidas com Flask, é comum utilizar o cliente de testes fornecido pelo próprio framework. Esse cliente permite simular requisições HTTP à aplicação sem a necessidade de iniciar um servidor real.

Um exemplo simplificado de teste para uma rota Flask seria:

```
1 def test_homepage(client):  
2     response = client.get("/")  
3     assert response.status_code == 200
```

Nesse caso, o teste verifica se a página inicial da aplicação responde corretamente com o código HTTP 200, indicando sucesso na requisição.

Testes em Aplicações Flask

Aplicações Flask podem ser testadas utilizando a extensão `pytest-flask`, que fornece recursos específicos para simular requisições HTTP.

Uma estrutura comum de testes inclui um arquivo chamado `conftest.py`, responsável por configurar objetos reutilizáveis chamados **fixtures**.

Exemplo:

```
1 import pytest  
2 from delivery.app import create_app  
3  
4 @pytest.fixture  
5 def app():  
6     return create_app()
```

Essa configuração permite que o objeto da aplicação esteja disponível automaticamente em todos os testes.

Simulação de Requisições

Com o PyTest-Flask, também é possível simular requisições HTTP utilizando um cliente de teste:

```
1 def test_not_found(client):  
2     response = client.get("/url-inexistente")
```

```
3 assert response.status_code == 404
```

Esse tipo de teste permite verificar se rotas, respostas e códigos HTTP estão funcionando corretamente.

Cobertura de Testes

Outra prática importante é medir a **cobertura de testes**, ou seja, quantas partes do código estão sendo exercitadas pelos testes.

Isso pode ser feito com a extensão:

```
1 pytest-cov
```

Executando:

```
1 pytest --cov=delivery
```

O relatório indica quais módulos foram testados e quais ainda precisam de cobertura adicional.

Considerações Finais

Nesta unidade foram apresentadas práticas fundamentais para desenvolvimento profissional de aplicações Python:

- organização do projeto como pacote instalável;
- gerenciamento de dependências;
- automação de tarefas de desenvolvimento;
- criação de testes automatizados.

Essas técnicas são amplamente utilizadas em projetos reais e contribuem para tornar aplicações mais robustas, reutilizáveis e fáceis de manter.

Nos próximos capítulos, esses conceitos serão aplicados continuamente à evolução da aplicação, garantindo qualidade e confiabilidade ao longo do desenvolvimento.

Referências Bibliográficas

- [1] IPython Development Team. *IPython: Interactive Computing in Python*. 2024. Acesso em: 9 mar. 2026. Disponível em: <<https://github.com/ipython/ipython>>.
- [2] Python Software Foundation. *Black: The Uncompromising Python Code Formatter*. 2024. Acesso em: 9 mar. 2026. Disponível em: <<https://github.com/psf/black>>.
- [3] Flake8 Development Team. *Flake8: Tool for Style Guide Enforcement in Python*. 2024. Acesso em: 9 mar. 2026. Disponível em: <<https://github.com/PyCQA/flake8>>.
- [4] Pytest Development Team. *pytest: Simple and Powerful Testing with Python*. 2024. Acesso em: 9 mar. 2026. Disponível em: <<https://github.com/pytest-dev/pytest>>.
- [5] pytest-dev. *pytest-cov: Coverage Plugin for pytest*. 2024. Acesso em: 9 mar. 2026. Disponível em: <<https://github.com/pytest-dev/pytest-cov>>.
- [6] pytest-dev. *pytest-flask: Pytest Plugin for Flask Applications*. 2024. Acesso em: 9 mar. 2026. Disponível em: <<https://github.com/pytest-dev/pytest-flask>>.